RoommateRadar - Complete Project Documentation

---

1. Project Overview

What It Does

RoommateRadar is a full-stack web application that helps people find compatible roommates. It connects two types of users: **Seekers** (students or working professionals looking for a room) and **Hosts** (landlords or tenants offering a room). The platform uses a custom **compatibility scoring algorithm** to match seekers with listings based on lifestyle preferences, budget, and living habits.

Problem It Solves

Finding a compatible roommate is one of the most frustrating aspects of relocating to a new city. Existing platforms (OLX, Flat & Flatmates, Facebook groups) offer no way to filter by compatibility. People end up with roommates whose sleep schedules, cleanliness standards, or budgets don't align, leading to conflict and early move-outs.

RoommateRadar solves this by:

- Collecting detailed lifestyle preferences during onboarding
- Automatically scoring every listing against the seeker's profile (0-100%)
- Letting users compare shortlisted options side-by-side
- Unlocking private chat only after mutual interest (request accepted)
- Building trust through a community rating system

Target Users

User Type	Description
Seeker	Students, working professionals, or anyone looking for a room and a compatible roommate
Host	Landlords, flat owners, or existing tenants who have a room to offer
Admin	Platform moderator who manages users, listings, and monitors activity

---

2. Key Features

Authentication & Authorization

- JWT-based stateless authentication
- Password strength enforcement (uppercase, lowercase, number, special character)
- Role-based access control (Seeker, Host, Admin)
- Auto-logout on token expiration via Axios interceptor

Onboarding with Mandatory Checklist

- Gender, age, city, area/locality (mandatory)
- Occupation selection (Student, Working Professional, Freelancer, etc.)
- Lifestyle tags (Early Riser, Night Owl, Fitness Enthusiast, Pet Lover, etc.)
- Optional bio and phone number
- Profile photo upload via Cloudinary

Smart Compatibility Scoring (0-100)

A custom algorithm evaluates 9 criteria between a seeker's preferences and a listing/host:

Criteria	Weight	Logic

Budget	20 pts	Full match = 20, within 20% tolerance = 10
Gender preference	10 pts	Match or either is "any" = 10
Smoking	10 pts	Non-smoker or listing allows = 10
Pets	10 pts	No pets or listing allows = 10
Food preference	10 pts	Match or either is "any" = 10
Sleep schedule	10 pts	Match or either is "flexible" = 10
Noise level	10 pts	Exact match = 10, moderate (middle ground) = 5
Cleanliness	10 pts	10 minus (difference x 3), minimum 0
Work schedule	10 pts	Match = 10, hybrid (middle ground) = 5

Listing Management

- Hosts can create, edit, and delete room listings
- Photo uploads (up to 10 images, 5MB each) via Cloudinary
- Rich listing details: rent, deposit, room type, amenities, availability date
- GeoJSON location support for proximity-based features

Browse & Filter

- Filter by city, rent range, room type, amenities
- Sort by rent, match score, or date posted
- Pagination for large result sets
- Loading skeleton animations during fetch

Request System

- Seekers send interest requests to hosts with an optional message
- Duplicate request prevention (unique constraint on seeker + listing)
- Hosts can accept or reject from their dashboard
- Seekers can withdraw pending requests
- Accepted requests automatically unlock a private chat conversation

Real-Time Chat (Socket.io)

- Conversations are created and unlocked only when a host accepts a request
- Real-time message delivery using Socket.io rooms
- Read receipts (bulk update on message load)
- Auto-scroll to latest message
- Visual distinction between sent and received messages

Shortlist & Compare

- Heart icon to bookmark listings
- Multi-select from shortlist for side-by-side comparison
- Comparison table shows rent, room type, amenities, match score

Rating & Trust System

- Users can rate each other after interaction (1-5 stars)
- Optional review text and tags (clean, responsive, honest, friendly, punctual)
- Trust score auto-computed as the average of all received ratings
- One rating per (rater, ratee, listing) combination

Role-Based Dashboards

Dashboard	Tabs
Seeker	Overview, My Requests, Shortlist, Profile
Host	Overview, My Listings, Incoming Requests
Admin	Platform Stats, User Management, Listing Moderation

Admin Panel

- Platform-wide statistics (total users, seekers, hosts, active listings, pending/accepted requests)
- User search, filter by role, toggle active/suspended
- Listing moderation (toggle visibility)

Dark Theme

- Global dark theme (black background, white text)
- CSS override strategy for Tailwind utility classes
- Consistent across all pages, forms, modals, and scrollbars

---

3. Tech Stack

Frontend

Technology	Version	Purpose
React	18.2	UI library (component-based architecture)
Vite	5.0	Build tool (fast HMR, ES modules)
React Router	6.20	Client-side routing with role-based guards
Tailwind CSS	3.4 (CDN)	Utility-first CSS framework
Axios	1.6	HTTP client with interceptors for auth
Socket.io Client	4.7	Real-time WebSocket communication
Leaflet	1.9	Interactive map support

Backend

Technology	Version	Purpose
Node.js	25.x	JavaScript runtime
Express	5.2	Web framework
Mongoose	9.3	MongoDB ODM (schema validation, middleware)
Socket.io	4.8	Real-time bidirectional event-based communication
JWT (jsonwebtoken)	9.0	Stateless authentication
bcryptjs	3.0	Password hashing (12 salt rounds)
Multer	2.1	Multipart file upload handling
Cloudinary	2.9	Cloud image storage and transformation
Morgan	1.10	HTTP request logging

dotenv	17.3	Environment variable management

Database

Technology	Purpose
MongoDB Atlas	Cloud-hosted NoSQL database
Mongoose Schemas	6 collections: Users, Listings, Requests, Conversations, Messages, Ratings

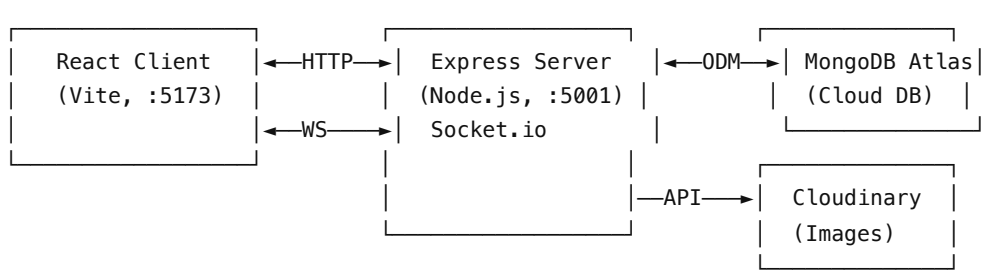
Why These Technologies?

- **React + Vite**: Fast development with hot module replacement; component reuse across dashboards
- **Express 5**: Async error handling improvements over v4; widely supported middleware ecosystem
- **MongoDB**: Flexible schema for user preferences and listing details; GeoJSON support for location
- **Socket.io**: Simplifies real-time chat without raw WebSocket management; automatic reconnection
- **Cloudinary**: Offloads image storage, provides on-the-fly transformations (crop, resize)
- **JWT**: Stateless auth scales easily; no server-side session storage needed

---

4. System Architecture

High-Level Overview



Data Flow

```
User Action (click, submit)
    │
    ▼
React Component → Axios Request (with JWT in header)
    │
    ▼
Express Router → Auth Middleware (verify JWT) → Role Check
    │
    ▼
Controller → Mongoose Model → MongoDB Atlas
    │
    ▼
JSON Response → React State Update → UI Re-render
```

Real-Time Chat Flow

```
Sender types message → POST /api/chat/conversations/:id/messages
    │
    ▼
Controller saves to DB → io.to(conversationId).emit('new_message', message)
```

```
    │
    ▼
Receiver's Socket.io client → onMessage handler → State update → UI
```

Request-to-Chat Unlock Flow

```
Seeker sends request → status: "pending"
    │
    ▼
Host clicks "Accept" → status: "accepted"
    │
    ▼
Backend auto-creates Conversation (isUnlocked: true)
    │
    ▼
Both users see conversation in /chat
```

---

5. Folder Structure

```
roomateradar/
├── backend/
│   ├── config/
│   │   ├── db.js                 # MongoDB connection setup
│   │   └── cloudinary.js         # Cloudinary SDK configuration
│   ├── middleware/
│   │   ├── auth.js               # JWT verification middleware
│   │   └── roleCheck.js          # Role-based access control
│   ├── models/
│   │   ├── User.js               # User schema + preferences subdocument
│   │   ├── Listing.js            # Room listing schema (GeoJSON location)
│   │   ├── Request.js            # Seeker-to-host interest requests
│   │   ├── Conversation.js       # Chat room between two users
│   │   ├── Message.js            # Individual chat messages
│   │   └── Rating.js             # User review and trust scoring
│   ├── controllers/
│   │   ├── authController.js     # Register, login, getMe
│   │   ├── userController.js     # Profile, preferences, avatar, shortlist
│   │   ├── listingController.js  # CRUD + photo upload + filtered browse
│   │   ├── requestController.js  # Send, respond, withdraw requests
│   │   ├── chatController.js     # Conversations + messages + Socket.io emit
│   │   ├── ratingController.js   # Create rating + auto-compute trust score
│   │   └── adminController.js    # Stats, user management, listing moderation
│   ├── routes/
│   │   ├── auth.js               # 3 endpoints
│   │   ├── users.js              # 6 endpoints
│   │   ├── listings.js           # 7 endpoints
│   │   ├── requests.js           # 5 endpoints
│   │   ├── chat.js               # 3 endpoints
│   │   ├── ratings.js            # 2 endpoints
│   │   └── admin.js              # 5 endpoints
│   ├── utils/
│   │   ├── compatibilityScore.js  # 9-criteria matching algorithm
│   │   └── generateToken.js       # JWT token generator
│   ├── server.js                 # Express + Socket.io + route mounting
```

```
│   └── .env                      # Environment variables
│
├── frontend/
│   ├── src/
│   │   ├── api/
│   │   │   └── axios.js          # Axios instance with JWT interceptor
│   │   ├── context/
│   │   │   ├── AuthContext.jsx     # Global auth state (user, login, logout)
│   │   │   └── SocketContext.jsx   # Socket.io connection lifecycle
│   │   ├── components/
│   │   │   ├── Navbar.jsx          # Role-aware navigation bar
│   │   │   ├── ProtectedRoute.jsx  # Auth guard for private routes
│   │   │   ├── ListingCard.jsx     # Listing preview card with match badge
│   │   │   ├── FilterPanel.jsx     # Search filters (city, rent, amenities)
│   │   │   ├── RequestButton.jsx   # Request modal with status tracking
│   │   │   ├── ChatWindow.jsx      # Real-time message panel
│   │   │   ├── MatchBadge.jsx      # Colored compatibility score pill
│   │   │   └── MapView.jsx         # Leaflet map component
│   │   ├── pages/
│   │   │   ├── Landing.jsx         # Public homepage with hero section
│   │   │   ├── Login.jsx           # Email/password login form
│   │   │   ├── Register.jsx        # Registration with password validation
│   │   │   ├── Onboarding.jsx      # 2-step profile + preferences setup
│   │   │   ├── Browse.jsx          # Listing grid with filters + pagination
│   │   │   ├── ListingDetail.jsx   # Full listing page with host info
│   │   │   ├── CreateListing.jsx   # Host listing creation form
│   │   │   ├── Chat.jsx            # Conversation sidebar + chat window
│   │   │   ├── Shortlist.jsx       # Saved listings with compare feature
│   │   │   ├── Compare.jsx         # Side-by-side listing comparison
│   │   │   └── Dashboard/
│   │   │       ├── SeekerDashboard.jsx   # 4-tab seeker control panel
│   │   │       ├── HostDashboard.jsx     # 3-tab host control panel
│   │   │       └── AdminDashboard.jsx    # 3-tab admin control panel
│   │   ├── utils/
│   │   │   └── formatters.js       # Currency, date, distance formatters
│   │   ├── App.jsx                 # Route definitions + providers
│   │   ├── main.jsx                # React DOM entry point
│   │   └── index.css               # Global dark theme overrides
│   ├── index.html                  # HTML shell with Tailwind CDN
│   ├── vite.config.js              # Vite configuration
│   └── .env                        # VITE_API_URL
│
└── DOCUMENTATION.md                # This file
```

---

6. How It Works (Step-by-Step Flow)

Flow 1: New User Registration

```
1. User visits /register
2. Fills name, email, password (validated: uppercase + lowercase + number + special char), role
3. Frontend POST → /api/auth/register
4. Backend hashes password (bcrypt, 12 rounds), creates User document
5. JWT token generated and returned
6. Token stored in localStorage, user redirected to /onboarding
```

Flow 2: Onboarding

```
Step 1 — Profile:
1. User fills mandatory checklist: gender, age, city, area, occupation, lifestyle tags
2. Optionally writes bio and uploads profile photo
3. Frontend POST → /api/users/upload-avatar (Cloudinary)
4. Frontend PUT → /api/users/profile (bio assembled from checklist + free text)

Step 2 — Preferences:
1. User sets food, sleep, smoking, drinking, pets, gender, cleanliness, noise, work schedule, budget
2. Frontend PUT → /api/users/preferences
3. isProfileComplete set to true
4. User redirected to dashboard based on role
```

Flow 3: Host Creates a Listing

```
1. Host navigates to /create-listing
2. Fills title, address, rent, deposit, room type, amenities, rules
3. Frontend POST → /api/listings (creates listing)
4. Optionally uploads photos → POST /api/listings/:id/photos (Cloudinary)
5. Listing appears in host's dashboard and public browse page
```

Flow 4: Seeker Browses & Matches

```
1. Seeker visits /browse
2. Frontend GET → /api/listings (with optional filters)
3. Backend fetches listings, computes compatibility score for each
4. Returns listings sorted by match %, with score attached
5. Seeker sees listing cards with colored match badges (green/yellow/red)
```

Flow 5: Request → Accept → Chat

```
1. Seeker clicks "Request to Stay" on a listing
2. Frontend POST → /api/requests (with optional message)
3. Host sees request in dashboard → clicks "Accept"
4. Frontend PUT → /api/requests/:id { status: 'accepted' }
5. Backend creates Conversation { isUnlocked: true, participants: [seeker, host] }
6. Both users now see the conversation in /chat
7. Messages sent via POST + emitted via Socket.io in real-time
```

Flow 6: Rating

```
1. After interaction, either user visits the other's profile
2. Submits rating (1–5 stars) + optional review + tags
3. Backend upserts rating, recalculates ratee's trustScore as average
4. Trust score displayed on user profiles
```

---

7. Important Code Concepts

JWT Authentication Flow

```
Register/Login → Server generates JWT with user ID → Client stores in localStorage
    ↓
Every API call → Axios interceptor attaches "Authorization: Bearer <token>"
    ↓
Server middleware (auth.js) → jwt.verify() → attaches req.user → next()
    ↓
401 response → Axios response interceptor → clears token → redirects to /login
```

Role-Based Middleware (Layered)

```
// Applied in sequence:
router.use(protect);                    // Step 1: Is the user logged in?
router.use(roleCheck('admin'));         // Step 2: Does the user have this role?
// Only then does the request reach the controller
```

Compatibility Scoring Algorithm

The algorithm in `utils/compatibilityScore.js` uses a **weighted additive model**:

- 9 criteria, each worth 10 points (budget is worth 20)
- Maximum possible score: 100
- Criteria are split into two groups:
  - **Listing-based** (budget, gender, smoking, pets) — always evaluable
  - **Host preference-based** (food, sleep, noise, cleanliness, work) — requires host to have completed onboarding; if missing, neutral 50 points awarded
- Partial credit logic for near-matches (e.g., budget within 20% tolerance, moderate noise as middle ground)

Socket.io Room-Based Chat

```
Client connects → server assigns socket.id
Client joins room → socket.join(conversationId)
Message sent → io.to(conversationId).emit('new_message', message)
Only participants in the room receive the event
Client leaves room → socket.leave(conversationId)
```

Request Duplication Prevention

The Request model has a unique compound index on `(seeker, listing)`. If a seeker tries to send a second request to the same listing, MongoDB throws error code `11000`, which the controller catches and returns a friendly "Already sent a request" message.

Cloudinary Upload Pattern

```
Multer (memory storage) → Buffer in req.file
    ↓
Create readable stream from buffer → Pipe to Cloudinary upload_stream
    ↓
Cloudinary returns secure_url → Stored in MongoDB document
```

Dark Theme Strategy

Instead of modifying 30+ component files, a single `index.css` overrides Tailwind's light utility classes globally:

```
.bg-white    { background-color: #111111 !important; }
.text-gray-800 { color: #f0f0f0 !important; }
/* ...40+ overrides covering backgrounds, borders, text, inputs, shadows */
```

This approach is fast to implement and easy to revert.

---

8. Setup & Installation

Prerequisites

- Node.js (v18 or higher)
- MongoDB Atlas account (free tier works)
- Cloudinary account (free tier works)

Step 1: Clone the Repository

```
git clone <repository-url>
cd roomateradar
```

Step 2: Backend Setup

```
cd backend
npm install
```

Create `backend/.env`:

```
PORT=5001
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/roomateradar
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRES_IN=7d
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
CLIENT_URL=http://localhost:5173
```

> ***Note:*** *If on macOS, port 5000 is often occupied by AirPlay Receiver. Use 5001 instead.*

Step 3: Frontend Setup

```
cd ../frontend
npm install
```

Create `frontend/.env`:

```
VITE_API_URL=http://localhost:5001/api
```

Step 4: MongoDB Atlas Network Access

1. Go to cloud.mongodb.com → Network Access
2. Add your current IP address (or `0.0.0.0/0` for development)

Step 5: Run the Application

In two separate terminals:

```
# Terminal 1 — Backend
cd backend
node server.js
# Should print: MongoDB connected, Server running on port 5001

# Terminal 2 — Frontend
cd frontend
npm run dev
# Should print: Local: http://localhost:5173/
```

Step 6: Create an Admin User

1. Register a regular account at `http://localhost:5173/register`
2. In MongoDB Atlas → Browse Collections → `roomateradar.users`
3. Find your user → Edit → change `role` from `"seeker"` to `"admin"`

---

9. Challenges & Solutions

Challenge 1: `next is not a function` in Mongoose Pre-Save Hook

Problem: Mongoose 9.x removed the `next` callback from async pre-save hooks. The User model was calling `next()` in the password hashing hook, causing a crash on every registration.

Solution: Changed the hook from `async function(next) { ... next() }` to `async function() { ... return }`. The async function's resolved promise signals Mongoose to proceed.

Challenge 2: Port 5000 Occupied on macOS

Problem: macOS Monterey and later use port 5000 for AirPlay Receiver (`ControlCenter` process), causing `EADDRINUSE` on backend startup.

Solution: Changed the backend to port 5001 and updated both `.env` files accordingly. This is a common macOS development issue.

Challenge 3: MongoDB Atlas IP Whitelist

Problem: Backend could connect locally but Atlas rejected connections with "Could not connect to any servers" because the developer's IP was not whitelisted.

Solution: Added the current public IP to Atlas → Network Access. For development, `0.0.0.0/0` (allow all) can be used temporarily.

Challenge 4: Dark Theme Without Refactoring 30+ Components

Problem: All components were built with Tailwind's default light classes (`bg-white` , `text-gray-800`). Converting each file individually would be time-consuming and error-prone.

Solution: A single `index.css` file overrides Tailwind utility classes using `!important` . This inverts the entire theme globally without touching component code.

Challenge 5: Chat Should Only Unlock After Acceptance

Problem: Allowing anyone to message a host would invite spam. Chat needs to be gated behind a verified interaction.

Solution: Conversations are only created in the `respondToRequest` controller when `status: 'accepted'` is set. The Conversation document has an `isUnlocked` flag, and the chat query filters by `isUnlocked: true` .

Challenge 6: Preventing Spam Registrations

Problem: Bots or low-effort signups could pollute the platform with incomplete profiles.

Solution: Mandatory onboarding checklist (gender, age, city, area, occupation, lifestyle tags) that must be completed before accessing any feature. Password validation enforces complexity. The `isProfileComplete` flag gates access to Browse and Dashboard.

---

10. Improvements / Future Scope

Near-Term Enhancements

- **Email verification**: Send OTP or magic link to confirm email before activation
- **Push notifications**: Browser notifications for new messages and request updates
- **Map-based browse**: Show listings on a Leaflet map with proximity radius filter
- **Advanced search**: Full-text search on listing titles and descriptions
- **Photo gallery**: Lightbox viewer for listing images

Medium-Term Features

- **Video verification**: Short video call feature for identity verification
- **Mutual matching**: Both seeker and host must express interest before unlocking chat
- **Lease/agreement templates**: Generate rental agreements within the platform
- **Payment integration**: Security deposit collection via Razorpay/Stripe
- **Mobile app**: React Native version sharing the same backend API

Long-Term Vision

- **Machine learning matching**: Train a model on successful roommate pairings to improve scoring
- **Multi-city expansion**: City-specific landing pages with local partnerships
- **Roommate analytics**: Dashboard showing preference trends, average rents, popular areas
- **Verified landlord program**: Background-checked hosts with a trust badge
- **Internationalization (i18n)**: Multi-language support for global reach

---

11. Summary (1-2 Minute Explanation)

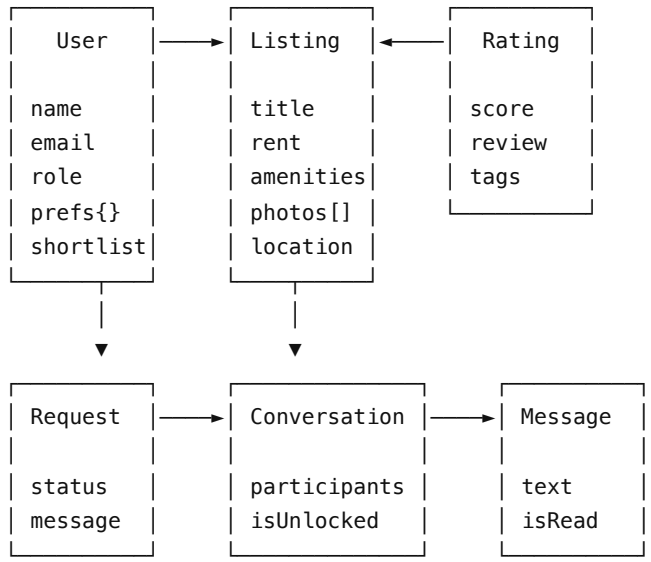
> ***RoommateRadar*** *is a full-stack web application I built to solve the problem of finding compatible roommates. It connects people looking for rooms (seekers) with those offering rooms (hosts).*
>
> *The platform stands out with its* ***smart compatibility scoring algorithm*** *that evaluates 9 lifestyle factors — budget, sleep schedule, cleanliness, food habits, noise preference, and more — to generate a 0-100 match percentage for every listing.*
>
> *On the tech side, I used* ***React 18 with Vite*** *for the frontend,* ***Node.js with Express 5*** *for the backend,* ***MongoDB Atlas*** *for the database, and* ***Socket.io*** *for real-time chat. Images are handled through* ***Cloudinary****, and the entire UI uses a* ***dark theme*** *with Tailwind CSS.*
>
> *The application supports* ***three user roles*** *— Seeker, Host, and Admin — each with their own dashboard. Key features include a mandatory onboarding checklist, listing creation with photo uploads, a request system that unlocks private chat only after acceptance, shortlisting with side-by-side comparison, and a rating system that builds user trust scores.*
>
> *I also built an* ***admin panel*** *for platform moderation — managing users, monitoring listings, and viewing platform-wide statistics.*
>
> *The project has* ***31 API endpoints*** *across 7 route groups,* ***6 MongoDB collections****,* ***13 frontend pages****, and* ***8 reusable components****. It follows a clean MVC architecture on the backend and a context-based state management pattern on the frontend.*

API Endpoint Reference

Group	Method	Endpoint	Auth	Description
Auth	POST	`/api/auth/register`	Public	Register new user
	POST	`/api/auth/login`	Public	Login
	GET	`/api/auth/me`	Protected	Get current user
Users	GET	`/api/users/:id`	Public	Get user profile
	PUT	`/api/users/profile`	Protected	Update profile
	PUT	`/api/users/preferences`	Protected	Update preferences
	POST	`/api/users/upload-avatar`	Protected	Upload avatar
	PUT	`/api/users/shortlist/:id`	Protected	Toggle shortlist
	GET	`/api/users/shortlist`	Protected	Get shortlist
Listings	GET	`/api/listings`	Protected	Browse listings
	POST	`/api/listings`	Host	Create listing
	GET	`/api/listings/my`	Host	My listings
	GET	`/api/listings/:id`	Public	Listing detail
	PUT	`/api/listings/:id`	Host/Admin	Update listing
	DELETE	`/api/listings/:id`	Host/Admin	Delete listing
	POST	`/api/listings/:id/photos`	Host	Upload photos
Requests	POST	`/api/requests`	Seeker	Send request
	GET	`/api/requests/my`	Seeker	My requests
	GET	`/api/requests/incoming`	Host	Incoming requests
	PUT	`/api/requests/:id`	Host	Accept/reject
	DELETE	`/api/requests/:id`	Seeker	Withdraw request
Chat	GET	`/api/chat/conversations`	Protected	My conversations
	GET	`/api/chat/conversations/:id/messages`	Protected	Get messages
	POST	`/api/chat/conversations/:id/messages`	Protected	Send message
Ratings	POST	`/api/ratings`	Protected	Rate a user
	GET	`/api/ratings/user/:id`	Public	Get user ratings
Admin	GET	`/api/admin/stats`	Admin	Platform statistics
	GET	`/api/admin/users`	Admin	List/search users
	PUT	`/api/admin/users/:id/toggle`	Admin	Suspend/activate user
	GET	`/api/admin/listings`	Admin	All listings

	PUT	`/api/admin/listings/:id/toggle`	Admin	Toggle listing visibility

Database Schema Summary



Generated for RoommateRadar v1.0 — March 2026